UNIVERSIDAD MAYOR DE SAN ANDRÉS

Carrera de Informática | Computación en la Nube

Funcionamiento del sistema de certificación académica y su Blockchain

Guía explicativa para comprender y exponer el prototipo

25 de octubre de 2026

El sistema permite emitir certificados académicos, generar un PDF con QR, registrar huellas digitales en un contrato inteligente, verificar el documento y revocarlo. Esta guía explica qué hace cada tecnología y cómo trabajan juntas, con ejemplos y respuestas para una exposición.

La implementación funciona localmente con React, Node.js, SQLite y una red EVM de Hardhat. Los documentos tienen fines académicos y contienen la leyenda Prototipo académico - Sin validez oficial.

1 Qué se encuentra implementado

- Inicio de sesión con roles de administrador y emisor institucional.
- Registro, edición y búsqueda de estudiantes; gestión y autorización de usuarios.
- Emisión de certificados PDF con código único y QR, individual o por CSV.
- Verificación pública por código, enlace QR o carga del archivo PDF.
- Revocación con justificación y conservación del historial.
- Dashboard, bitácora y persistencia de datos entre reinicios del sistema.

Cómo está organizada la explicación

Tema	Página
Tecnologías y responsabilidad de cada componente	2
SHA-256 y ejemplos de integridad	3
Contrato inteligente y permisos	4
Emisión y confirmación de una transacción	5
Verificación pública y revocación	6
Datos privados y límites de la red local	7
Preguntas para exposición y uso del proyecto	8

2 Qué es cada tecnología del sistema

Los nombres utilizados no representan lo mismo: algunos son lenguajes, otros son bibliotecas, herramientas o archivos del proyecto. Cada componente resuelve una parte del trabajo.

Componente	Qué es	Para qué se utiliza
React	Biblioteca de JavaScript	Construye pantallas, formularios y resultados de verificación.
Vite	Herramienta de desarrollo	Sirve la interfaz y compila sus archivos.
Node.js y Express	Entorno de ejecución y framework	Ejecutan el backend, validan datos y exponen la API REST.
SQLite	Motor de base de datos	Guarda usuarios, estudiantes, certificados y operaciones.
SHA-256	Algoritmo criptográfico de hash	Calcula las huellas de datos y archivos.
Solidity	Lenguaje de programación	Permite escribir las reglas del contrato inteligente.
AcademicCertRegistry.sol	Archivo de código fuente	Contiene el contrato de registro y revocación.
ethers.js	Biblioteca de JavaScript	Conecta el backend con el nodo y firma transacciones.
Hardhat	Herramienta de desarrollo EVM	Compila y prueba contratos y ejecuta la red local.

2 1 La relación entre Solidity y el archivo del contrato

Solidity es el lenguaje; AcademicCertRegistry.sol es el archivo escrito en ese lenguaje. La extensión .sol identifica ese tipo de código fuente. De forma similar, un archivo .jsx puede contener la interfaz escrita con React, pero el archivo y la biblioteca son cosas distintas.

2 2 La relación entre ethers y Hardhat

ethers.js es el medio de comunicación del backend. Hardhat ofrece el nodo local que recibe las solicitudes y ejecuta el contrato. No son dos contratos ni dos bases de datos. El navegador llama a la API y la API utiliza ethers para consultar o modificar el registro de la cadena.

La EVM es la máquina virtual que interpreta las instrucciones compiladas del contrato. Hardhat ejecuta una EVM de desarrollo compatible con el tipo de contrato que después puede desplegarse en una red Ethereum de pruebas.

3 Qué es SHA 256 y cómo detecta cambios

SHA-256 recibe una secuencia de bytes y calcula una huella de 256 bits. Al escribirla en hexadecimal se representa con 64 caracteres. Una entrada puede ser un texto corto, un conjunto de datos o los bytes completos de un PDF.

3 1 Ejemplo calculado con un texto

Los siguientes valores se calcularon con los textos exactos hola y Hola en UTF-8. Cambiar solamente la mayúscula modifica la huella.

Texto de entrada: hola

```
b221d9dbb083a7f33428d7c2a3c3198ae925614d70210e28716ccaa7cd4ddb79
```

Texto de entrada: Hola

```
e633f4fc79badea1dc5db970cf397c8248bac47cc3acf9915ba60b5d76b0e88f
```

Los mismos bytes producen siempre la misma huella. Si las huellas son diferentes, los archivos o datos no son idénticos. Aunque existen colisiones como posibilidad matemática, SHA-256 está diseñado para que encontrar una de forma práctica sea extremadamente difícil.

3 2 Los dos hashes del certificado

Huella	Entrada utilizada	Qué permite comprobar
Hash de los datos	JSON canónico con información académica, UUID, código y salt.	Que los valores conservados correspondan al registro emitido.
Hash del PDF	Todos los bytes del documento generado.	Que el archivo cargado coincida con el PDF registrado.

El JSON canónico utiliza claves ordenadas y una serialización uniforme. El salt es un valor aleatorio de 16 bytes por certificado y se guarda fuera de la cadena. Hace menos viable reconstruir la huella probando solamente datos personales conocidos. No reemplaza los controles de acceso a esos datos.

3 3 Hash no significa cifrado ni firma del PDF

El cifrado permite recuperar un contenido con la clave adecuada; un hash no contiene una copia recuperable del PDF. La firma de la transacción demuestra qué wallet autorizó la operación. En este prototipo no se incorpora una firma institucional oficial al documento.

El hash de la transacción también es distinto: identifica la operación enviada a la red y se calcula con mecanismos de Ethereum, como Keccak-256. No debe confundirse con las dos huellas SHA-256 del certificado.

4 Qué hace el contrato inteligente

AcademicCertRegistry.sol es el programa que establece las reglas de registro y revocación. El backend le solicita operaciones, pero el contrato comprueba los permisos y las condiciones durante su ejecución en la EVM.

4 1 Del archivo a un contrato desplegado

1. Se escribe el código fuente del contrato en Solidity.

2. El compilador solc lo transforma en bytecode, que entiende la EVM, y genera una ABI, que describe cómo llamar a sus funciones.

3. Un script envía una transacción de despliegue a la red.

4. La red crea una instancia del contrato con una dirección. ethers utiliza esa dirección y su ABI para comunicarse con él.

4 2 Funciones implementadas

Función	Qué hace	Condición principal
issue	Registra certHash y pdfHash y emite un evento.	Cuenta con ISSUER_ROLE, hashes no vacíos y registro no duplicado.
verify	Devuelve existencia, revocación, emisor, fecha y hash PDF.	Consulta pública de solo lectura.
revoke	Guarda la revocación y emite un evento.	Emisor original autorizado o administrador del contrato; registro vigente.

4 3 Ejemplo tomado del contrato

```
require(certificates[certHash].issuer == address(0),
        "AlreadyRegistered");
```

Esta condición exige que no exista todavía un emisor asociado al hash. Si el hash ya se registró, la operación se revierte. La interfaz no puede omitir esa regla simplemente cambiando un botón.

4 4 Roles web y roles de Blockchain

ADMIN y EMISOR son permisos de usuarios de la aplicación. DEFAULT_ADMIN_ROLE e ISSUER_ROLE son permisos de direcciones del contrato. Son dos controles distintos. En esta versión, los emisores web autorizados trabajan mediante una wallet institucional del backend; no necesitan una wallet personal ni MetaMask.

Los eventos CertificateIssued y CertificateRevoked dejan evidencia de operaciones en los recibos y registros del nodo. No incluyen los nombres ni los documentos de identidad de los estudiantes.

5 Cómo se emite y confirma un certificado

Ejemplo de uso: un emisor desea certificar que Ana aprobó Computación en la Nube. Primero registra a la estudiante y después completa el formulario de emisión. Lo siguiente ocurre en el backend.

5 1 Proceso completo

1. La API valida la sesión, los permisos y los datos del formulario.

2. Obtiene los datos del estudiante, genera UUID y código y crea un salt aleatorio.

3. Construye la representación canónica y calcula su SHA-256. Guarda un registro pendiente en SQLite.

4. Genera el PDF con el código, el UUID en metadatos, el QR y la leyenda de prototipo. Lo guarda de forma privada y calcula su SHA-256.

5. ethers prepara la llamada issue con ambos hashes y firma una transacción con la wallet institucional.

6. El nodo de Hardhat recibe la transacción, ejecuta el contrato y la incorpora a un bloque si se cumplen las reglas.

7. El backend espera el recibo y comprueba que la ejecución haya sido exitosa. Entonces actualiza el estado a emitido, guarda la bitácora y habilita la descarga.

5 2 Qué es una wallet una transacción y un bloque

Concepto	Explicación en el sistema
Wallet	Cuenta con dirección y clave privada. La clave autoriza la firma de operaciones y permanece en el backend.
Transacción	Solicitud firmada para cambiar el estado de la cadena, por ejemplo registrar o revocar un certificado.
Bloque	Agrupación de transacciones procesadas. El nodo local produce bloques automáticamente en su configuración de desarrollo.
Recibo	Resultado de la ejecución con estado, bloque y eventos. Permite confirmar si la operación tuvo éxito.
Gas	Medida del trabajo de ejecución de la EVM. Hardhat utiliza fondos de desarrollo, sin gasto de dinero real.

5 3 Qué ocurre si falla la operación

Generar el PDF no basta para declarar una emisión exitosa. Si la transacción no se completa, el registro queda en error y no se entrega como emitido. Como SQLite y la cadena no comparten una transacción atómica, la aplicación conserva evidencias y permite reconciliar estados interrumpidos contra el contrato y el archivo privado.

6 Cómo se verifica y se revoca un certificado

6 1 Verificación por código o QR

El QR es un enlace al portal público con el código del certificado. No contiene el PDF ni representa una firma digital. Al abrirlo, la API busca el registro, recalcula la huella de los datos y consulta verify en el contrato para comprobar existencia y revocación.

Copiar un QR no vuelve auténtico otro documento. El enlace seguirá mostrando los datos del certificado al que pertenece. Para comprobar los bytes del archivo concreto se utiliza la verificación mediante carga del PDF.

6 2 Verificación del archivo original

1. El usuario selecciona su PDF sin necesidad de iniciar sesión.

2. El backend lee el UUID incluido en los metadatos y localiza el certificado.

3. Calcula SHA-256 del archivo completo y lo contrasta con el hash registrado.

4. Consulta la vigencia en el contrato y presenta el resultado.

Resultado	Qué significa
Válido	Los controles de integridad coinciden, existe en cadena y no está revocado.
Revocado	El registro existe, pero se anuló su vigencia.
No encontrado	No existe un registro verificable para ese código o archivo.
Modificado o no coincidente	El archivo o los datos no corresponden a la huella esperada.
Error de conexión	No fue posible consultar Blockchain; no se confirma la validez.

Una copia exacta del PDF mantiene su huella. Volver a guardarlo, optimizarlo o imprimirlo a otro PDF puede cambiar sus bytes aunque conserve la misma apariencia. Un resultado no coincidente demuestra una diferencia, pero por sí solo no demuestra intención de fraude.

6 3 Revocación con conservación del historial

El emisor propietario o administrador introduce un motivo y solicita la revocación. ethers envía la transacción revoke; el contrato comprueba las condiciones y registra la fecha. Después de la confirmación, la base se actualiza y las consultas públicas muestran revocado.

El PDF no se borra ni se modifica. Puede conservar su integridad y, al mismo tiempo, estar revocado. Si se necesita corregir un certificado, se revoca el anterior y se emite uno nuevo. La justificación se guarda en SQLite, no dentro de Blockchain.

7 Qué se guarda y qué garantiza la red local

7 1 Separación de información

Información	Dentro de Blockchain	Ubicación utilizada
Huella de datos y huella PDF	Sí	Contrato AcademicCertRegistry
Wallet emisora y fechas del contrato	Sí	Registro y eventos del contrato
Nombres y documento de identidad	No	SQLite y copia privada de datos
Contraseñas de usuarios	No	SQLite con hash bcrypt
Salt y motivo de revocación	No	Registro privado en SQLite
Archivo PDF	No	Carpeta privada runtime/pdf
Clave privada de firma	No	Configuración del backend

La cadena no guarda los documentos completos. La base de datos sigue siendo necesaria para localizar certificados y presentar la información académica. El almacenamiento privado conserva el archivo que se entregará al usuario.

7 2 Qué significa una red real de Hardhat

Hardhat ejecuta bytecode del contrato en una EVM de desarrollo. Las transacciones se firman, ejecutan y reciben confirmación; no son filas inventadas en SQLite. Sin embargo, el nodo está bajo control de la computadora local y se puede reiniciar o restablecer. Esta configuración no ofrece la misma descentralización ni las garantías de permanencia de una red pública.

Para recuperar la demostración entre reinicios, el lanzador reproduce en una red local vacía las transacciones firmadas que conserva en runtime/chain.jsonl. Esto reconstruye el estado mediante ejecución EVM. Los hashes de transacción se conservan; las fechas y hashes de bloque pueden variar. Debe mantenerse un respaldo coherente de runtime.

7 3 Qué demuestra y qué no demuestra el registro

El registro permite comprobar que una wallet autorizada registró unas huellas y cuál es su estado de revocación en la red consultada. No demuestra por sí solo que el estudiante haya aprobado una materia: esa información debe validarla la institución antes de emitir. Tampoco convierte un documento de prueba en un título oficial.

Migrar a Sepolia exigiría desplegar el contrato y configurar RPC y wallets de prueba. Llevar el backend a la nube añadiría HTTPS, gestión de secretos y almacenamiento apropiado. Publicar una página web no cambia automáticamente la red Blockchain. Esas etapas no forman parte del despliegue local validado.

8 Preguntas para la exposición y uso del proyecto

Cómo explicar el proyecto en un minuto

Nuestro sistema permite emitir certificados PDF y verificar su autenticidad y vigencia. El backend calcula dos huellas SHA-256, una de los datos con un salt y otra del PDF. Con ethers envía esas huellas al contrato AcademicCertRegistry, escrito en Solidity y ejecutado en una red local de Hardhat. Al verificar se consulta el contrato y, si se carga un PDF, se compara también su huella. La revocación se registra mediante otra transacción. Los datos personales y documentos permanecen fuera de la cadena.

Preguntas frecuentes

¿Por qué utilizar Blockchain si ya existe SQLite?

SQLite administra personas, archivos y operaciones. El contrato aplica las reglas del registro criptográfico y permite consultar la revocación. En este prototipo ambos recursos son locales; una red pública aportaría una separación de confianza que aquí todavía no existe.

¿Se necesita dinero real o MetaMask?

No para esta demostración. Hardhat proporciona cuentas y fondos de desarrollo y el backend firma las operaciones. En una red externa se necesitan fondos y configuración de prueba adecuados.

¿Por qué el sistema puede mostrar Failed to fetch?

La interfaz no consigue completar la conexión con el servidor. Deben mantenerse activos el frontend y la API. No es, por sí mismo, una señal de contraseña incorrecta. La aplicación presenta un mensaje más claro cuando el backend no responde.

¿Qué se probó realmente?

La validación documentada del 22 de octubre de 2026 aprobó ocho pruebas de contrato y quince casos de integración: permisos, emisión, PDF, lectura del QR, modificación, revocación, fallos de conexión y persistencia. No equivale a una auditoría externa y no incluyó un despliegue en nube o Sepolia.

Inicio local y documentos de apoyo

```
npm install
npm run setup
npm run dev
```

Los dos primeros comandos corresponden a la preparación inicial. Para volver a ejecutar la instalación existente basta npm run dev desde la carpeta del proyecto. Abrir http://localhost:5173 y mantener la terminal activa. Los documentos generados siguen siendo un prototipo sin validez oficial.

Fuentes del proyecto: contrato contracts/AcademicCertRegistry.sol; módulos backend/blockchain.cjs y backend/certificates.cjs; README.md; docs/VALIDACION.md; y el informe integral entregado. El informe integral conserva la instalación detallada, los usuarios de demostración y las evidencias.